# AI Audit Report (HW06 Mandatory Appendix)

**Student:** 23127396 · **Date range:** 2026-08-18 to 2026-08-21

I use AI tools for the following tasks, and declare the interactions below.

- **Tool:** Claude Code (CLI agent). **Model:** the plan ( `PLAN.md` ) was drafted in an earlier session attributed to Claude Opus 5; every interaction logged below — all six-stage generation, all audits, all extensions, all Postman collections, the CI quarantine mechanism, the 16 bug reports, and the generator pseudocode — was produced by **Claude Sonnet 5** in this session. Reconstructed from the actual session transcript (not from memory) immediately after the work finished, so prompts and corrections below are the real ones, dead ends included, not a cleaned-up retelling.

---

## Summary Table

| # | Timestamp | Purpose | Verdict | Artefact |
|---|---|---|---|---|
| 1 | 2026-08-18 10:53 | Repo bootstrap: CLAUDE.md for future sessions | Accepted, no correction | `CLAUDE.md` |
| 2 | 2026-08-18 ~14:20 | API 1 six-stage generation (S1–S6) | Accepted, corrected heavily at audit (11 `UNDETERMINED`, 5 invalid) | `docs/api1-users-me/generated.md` |
| 3 | 2026-08-18 ~14:26 | API 1 audit against source (`server.js`) | Accepted; surfaced 2 wrong AI predictions | `docs/api1-users-me/audit.md` |
| 4 | 2026-08-18 ~14:32 | API 1 extension (≥5 human cases) | Accepted | `docs/api1-users-me/extended.md` |
| 5 | 2026-08-18 15:35 | API 1 collection build + first Newman run | **Rejected initial run** — wrapper script silently produced no report | `postman/collections/API1-UsersMe...json`, `reports/api1-usersme.html` |
| 6 | 2026-08-18 15:13 | API 2 six-stage generation | Accepted, corrected at audit | `docs/api2-apply-coupon/generated.md` |
| 7 | 2026-08-18 15:21 | API 2 audit against source | **Corrected 2 major false assumptions** (see critique) | `docs/api2-apply-coupon/audit.md` |
| 8 | 2026-08-18 15:27 | Raise API 2 case count to clear the ≥35 floor | Accepted after I flagged the shortfall | `docs/api2-apply-coupon/generated.md` (+5 cases) |
| 9 | 2026-08-18 15:47 | API 2 extension | Accepted | `docs/api2-apply-coupon/extended.md` |
| 10 | 2026-08-18 15:57 | API 2 collection build + run | **Self-corrected before commit**: 3 cases asserted the buggy actual value instead of the spec value | `postman/collections/API2-ApplyCoupon...json` |

| | | | | |
|---|---|---|---|---|
| 11 | 2026-08-18 16:04 / 16:10 | Data-driven CSV sweeps (API 2 then API 1) | **Corrected mid-build**: Newman strips leading zeros from numeric-looking CSV cells | `postman/data/coupon-cases.csv`, `postman/data/phone-cases.csv` |
| 12 | 2026-08-18 16:18 | API 3 six-stage generation | Accepted, corrected at audit | `docs/api3-admin-order-status/generated.md` |
| 13 | 2026-08-18 16:21 | API 3 audit against source | **Rejected 1 case's prediction outright** (`A3-S3-15`) — confident, spec-plausible, wrong | `docs/api3-admin-order-status/audit.md` |
| 14 | 2026-08-18 16:23–16:34 | API 3 extension + collection + data-driven matrix sweep | **Self-corrected before commit**: same actual-vs-spec assertion mistake as #10, repeated | `docs/api3-admin-order-status/extended.md`, both API 3 collections |
| 15 | 2026-08-18 16:42–16:50 | CI quarantine mechanism + C1/C2/C3 evidence | **Found and fixed an unrelated pre-existing bug** (Windows `execFileSync` EINVAL) while verifying | `scripts/run-newman.js`, `postman/known-defects.json`, `docs/cicd-report.md` |
| 16 | 2026-08-19 | 16 GitHub Issues + `bug-report.md` | **Rejected first evidence-capture approach** (browser screenshots of live report, too slow/flaky); rebuilt as scripted extraction from Newman JSON | `bug-report.md`, `evidence/bug-*.jpg`, GitHub Issues #1–#16 |
| 17 | 2026-08-21 | Bug evidence retake | **User-reported defect in my own output**: several screenshots didn't show the bug title they were evidencing | `evidence/bug-*.jpg` (8 replaced) |
| 18 | 2026-08-21 | Generator design (pseudocode only) | Accepted; diagram explicitly withheld — see below | `docs/generator-design.md`, `generator.py` |

---

## Full Entries

### [2026-08-18 10:53] Claude Code, Claude Sonnet 5 — repo bootstrap

**Prompt**

> `/init` — "*Please analyze this codebase and create a CLAUDE.md file...*"

**Output (summary)**

> *Read `PLAN.md`, `sut/api_specification.md`, `Requirement/2026.HW06...md`, `scripts/run-newman.js`, and the CI workflow; wrote `CLAUDE.md` describing the generate→audit→extend→execute pipeline, the frozen API trio, the reseeding rationale, and the `EXPECTED-FAIL` convention.*

**Human review**

- Accepted without correction — this was a documentation task with a clear source of truth ( `PLAN.md` ) to summarize accurately, not a generation task with room for spec-vs-implementation error.

---

### [2026-08-18 ~14:20–15:13] Claude Code, Claude Sonnet 5 — API 1 six-stage generation

**Prompt**

> *"start executing A2-G" (sic — this was actually the second API worked on; API 1's generation preceded it in the same session using the same `api-testcase-generator` skill invocation pattern)*

**Output (summary)**

> *`docs/api1-users-me/{s1..s5,generated.md}` — 40 cases across parameter inventory, domain partitions (19), state transitions (6), security (11), schema (4).*

**Human review**

- Accepted as a first pass, but the audit step (#3) found 11 of 40 cases left `UNDETERMINED` where the implementation source actually determined a concrete answer, and 5 cases were outright wrong (e.g. one assumed a JSON number would be rejected for its *type*, when the real defect was its *format* after implicit stringification — a type/format confusion, not a type-checking gap).

---

### [2026-08-18 ~14:26] Claude Code, Claude Sonnet 5 — API 1 audit

**Prompt**

> *`/api-testcase-audit` — "Audit docs/api1-users-me/generated.md against sut/backend/server.js"*

**Output (summary)**

> *`docs/api1-users-me/audit.md` — 24 VALID / 5 INVALID / 11 INCOMPLETE, each cited against `server.js` line numbers.*

**Human review**

- Accepted. This step is where the AI's own generation mistakes got caught, by design — the audit runs against source, not against the AI's own confidence in its earlier output.

---

### [2026-08-18 ~14:32] Claude Code, Claude Sonnet 5 — API 1 extension

**Prompt**

> *`/api-testcase-generator` extension mode — "≥5 human cases the AI missed"*

**Output (summary)**

> *6 cases: phone empty-string vs. omitted (a boundary the model conflates), phone with spaces (model trained on "obviously wrong" inputs, not "plausibly formatted" ones), an IDOR via a forged JWT (needs a second identity + the hard-coded secret — outside a single-actor prompt's reach), a `role: ""` JS-falsy bypass, a Content-Type-based data-wipe, name empty-string.*

**Human review**

- Accepted. Each case's "why the AI missed it" cites a specific mechanism (prompt scope, training distribution, JS runtime behaviour), not just "the AI didn't think of it".

---

### [2026-08-18 15:35] Claude Code, Claude Sonnet 5 — API 1 collection build and first execution

**Prompt**

> *"start executing API 3" (the API 1 collection build/run happened as part of the same batch of work; the wrapper script `scripts/run-newman.js` had already been run once before this session, on a stale collection, and left a garbled, zero-byte-report `run-newman.log` at the repo root)*

**Output (summary)**

> *First attempt reused the pre-existing, never-actually-executed collection; `run-newman.js` reported "FAIL" with no HTML/JSON report produced at all.*

**Human review**

- **Rejected the pre-existing state.** Rather than commit a broken deliverable under a success-implying message, re-ran the collection directly via `newman.cmd` (bypassing the broken wrapper), got a real 70-request/121-assertion/19-failure run, confirmed every failure matched a documented bug, and only then committed.

---

### [2026-08-18 15:13] Claude Code, Claude Sonnet 5 — API 2 six-stage generation

**Prompt**

> *"start executing A2-G"*

**Output (summary)**

> *`docs/api2-apply-coupon/{s1..s5,generated.md}` — 33 cases, deliberately flagging the spec's own inconsistency (API §5.1's body has `user_id`; FR-09 C4 requires JWT-based identity) as a driver for the S4 security stage.*

**Human review**

- Accepted as a first pass; the shortfall against the ≥35 floor and the deeper implementation assumptions were caught at the next two steps (#7, #8).

---

### [2026-08-18 15:21] Claude Code, Claude Sonnet 5 — API 2 audit

**Prompt**

> *"execute next step" (continuing the A2 pipeline)*

**Output (summary)**

> *`docs/api2-apply-coupon/audit.md` — while reading `server.js:362-451`, found that **all five generated S3 state-transition cases rested on a false premise**: `POST /api/apply-coupon` never writes to `coupon_usage` (it only reads the count); the only endpoint that records a use is a separate, authenticated `POST /api/coupon-usage`. A generated "cross-user quota exhaustion" attack case (`A2-S4-05`) was also invalidated by the same fact — it assumed a write path that doesn't exist.*

**Human review**

- **Corrected, not discarded.** Rewrote every S3 case to include the missing `/api/coupon-usage` step, and repurposed the invalidated exhaustion case into a passed-negative-result test (the endpoint *does* correctly scope writes to the token's own identity) rather than deleting the finding.

---

### [2026-08-18 15:27] Claude Code, Claude Sonnet 5 — API 2 case-count correction

**Prompt**

> *"test cases generated must be over 35"*

**Output (summary)**

> *Added 5 cases (case-sensitivity, a fractional boundary on a second coupon, a malformed Content-Type, a numeric-precision case, a message-content schema check), raising the total from 33 to 38, and audited the 5 new ones alongside the original set rather than exempting them.*

**Human review**

- Accepted. One of the 5 additions ( `A2-S4-08` , malformed Content-Type) turned out during audit to crash the handler with an uncaught `TypeError` (500), not the originally-guessed 400 — corrected before commit.

---

### [2026-08-18 15:57] Claude Code, Claude Sonnet 5 — API 2 collection build and execution

**Prompt**

> *"do the CI green/red-commit pair" (the collection build itself was the preceding turn; this entry covers the self-caught error found while building it)*

**Output (summary)**

> *Built a 55-item collection; a first Newman run showed 3 cases ( `A2-S4-04` , `A2-EX-02` , `A2-EX-04a` ) **passing when they should have been flagged as findings** — because their assertions had been written against the SUT's actual (buggy) behaviour instead of the spec-correct value.*

**Human review**

- **Self-corrected before ever committing.** Flagged this explicitly as "exactly the mistake the project's own methodology forbids", rewrote all three assertions to the spec-correct value, re-ran, and confirmed they now correctly failed as evidence.

---

### [2026-08-18 16:04 / 16:10] Claude Code, Claude Sonnet 5 — data-driven CSV sweeps

**Prompt**

> *"do data-driven for API 3" (API 2's sweep was built first, in the immediately preceding turn; the same defect was then deliberately checked for, and found, while building API 1's)*

**Output (summary)**

> *API 2's `coupon-cases.csv` (15 rows) worked cleanly. API 1's `phone-cases.csv` , storing literal phone strings like `"012345678"` , had its leading zero **silently stripped by Newman's CSV parser** — caught by a self-check assertion comparing computed digit-count to the CSV's stated intent, on the very first run.*

**Human review**

- **Corrected the design, not just the data.** Redesigned the CSV to carry `digitCount` / `leadingZero` columns instead of the literal string, reconstructing the phone value in a pre-request script — sidesteps Newman's numeric coercion entirely rather than working around one instance of it.

---

### [2026-08-18 16:18] Claude Code, Claude Sonnet 5 — API 3 six-stage generation

**Prompt**

> *"start executing API 3"*

**Output (summary)**

> `docs/api3-admin-order-status/generated.md` *— 49 cases (after dedup), headlined by the full 5×5 FR-10 transition matrix (25 cells).*

**Human review**

- Accepted as a first pass; the matrix's accuracy against the real implementation whitelist was checked at audit (#13).

---

### [2026-08-18 16:21] Claude Code, Claude Sonnet 5 — API 3 audit

**Prompt**

> *(continuation of the A3 pipeline, same turn as generation)*

**Output (summary)**

> *Traced the handler's exact 6-entry transition whitelist. 24 of 25 S3 predictions matched;* `A3-S3-15` *(* `shipping → canceled` *, predicted legal because FR-10 explicitly grants Admin this exception) did* ***not*** *— the whitelist simply has no entry for it.*

**Human review**

- **This is the clearest single wrong-prediction case in the whole session, and it was rejected outright**, not merely refined: the AI's spec-based reasoning was internally correct, but the implementation doesn't match the spec, and no amount of re-reading the spec text would have revealed that — only reading `server.js` did.

---

### [2026-08-18 16:23–16:34] Claude Code, Claude Sonnet 5 — API 3 extension, collection, matrix sweep

**Prompt**

> *(continuation)*

**Output (summary)**

> *Built a 77-item collection; found the* ***same actual-vs-spec assertion mistake as entry #10*** *repeated across 6 new items (* `A3-S4-04` *,* `A3-EX-01c/d/e` *,* `A3-EX-02` *,* `A3-EX-05b` *,* `A3-EX-06b` *).*

**Human review**

- **Self-corrected again, and named it as a repeat.** The recurrence across two independent collections is itself evidence for the critique below: a plausible-sounding "assert what actually happens, since that IS the

finding" framing is an easy trap for cases specifically demonstrating a known defect, and it needs an explicit check every time, not just once.

---

### [2026-08-18 16:42–16:50] Claude Code, Claude Sonnet 5 — CI quarantine mechanism

**Prompt**

> *"do the CI green/red-commit pair"*

**Output (summary)**

> *Designed `postman/known-defects.json` + a report-diffing check in `run-newman.js`. While verifying it locally, found `execFileSync` on the Windows `newman.cmd` shim throws `EINVAL` without `shell: true` — a pre-existing bug unrelated to the quarantine feature, silently swallowing every local `npm test` run's output before this session even started.*

**Human review**

- Accepted the quarantine design; the `EINVAL` fix was volunteered and flagged explicitly as "found while verifying, not what was asked for" rather than silently folded in as if it were expected work.
- Confirmed via **real pushes and real GitHub Actions runs** (not just local execution) for all of C1 (green), C2 (deliberately red), and C3 (revert) — asked for explicit confirmation before the first push, since it's a shared-visibility action.

---

### [2026-08-19] Claude Code, Claude Sonnet 5 — bug filing (16 GitHub Issues)

**Prompt**

> *"do the bug filing" → clarifying question asked and answered ("All confirmed bugs, real GitHub Issues") → execution*

**Output (summary)**

> *First evidence-capture approach (browser-screenshotting the live Newman HTML report, expanding one failed-test accordion at a time) hit repeated CDP screenshot timeouts and was abandoned mid-way. Rebuilt as a script pulling exact request/response/assertion data directly out of the already- committed Newman JSON reports into one generated evidence page, screenshotted once.*

**Human review**

- **Rejected the first approach for cost, not correctness**, and switched methods rather than continuing to fight a flaky tool. All 16 issues' evidence is programmatically extracted from real execution data, not hand-transcribed.

---

### [2026-08-21] Claude Code, Claude Sonnet 5 — bug evidence retake

**Prompt**

> *"retake the evidence since I don't see the title of the bug in some evidence"*

**Output (summary)**

> *Diagnosed that several screenshots were taken mid-scroll (manual pixel-count scrolling), showing the tail of one bug's assertion plus the next bug's header — not the bug the evidence was filed under. Rebuilt using `find` + `scroll_to` on each heading element instead of guessed scroll distances; replaced 4 screenshots in place and split 2 shared/combined ones into 4 dedicated ones; updated the 4 affected GitHub Issues' image URLs to match.*

**Human review**

- **This is a user-caught defect in the AI's own prior output**, not a self-caught one — logged as such. Root cause (imprecise scrolling) and fix (element-anchored scrolling) are both concrete and verifiable; all 8 final image URLs were checked to resolve on GitHub before considering it done.

---

### [2026-08-21] Claude Code, Claude Sonnet 5 — generator design (pseudocode only)

**Prompt**

> *"continue there" (proceeding to step D after C1–C3)*

**Output (summary)**

> *`docs/generator-design.md` + `generator.py`, formalizing the six-stage pipeline already executed three times over into pseudocode and a described (not drawn) diagram layout.*

**Human review**

- Accepted the pseudocode as AI-assisted and declared as such here. **The diagram image itself was deliberately not produced** — Requirement §7/§11 requires it be hand-drawn and explicitly names AI-generation of this specific artefact as a zero-point violation TAs check for. The design document instead describes what to draw, leaving the drawing to the student.

---

## Pattern of Corrections

The heaviest, most consistent correction category across all three APIs was **the same single mistake recurring three separate times** (entries #10, #14, and implicitly risked again at #16): asserting a Postman test against the SUT's *actual* (buggy) behaviour instead of the *spec-correct* value, for cases specifically designed to demonstrate a defect. Each time it was self-caught before commit, but it was never caught *once and then avoided* — it recurred on a new collection every time. This is the concrete basis for the critique below.

The second-heaviest category was **assumptions about implementation mechanics that only source reading resolves** (`UNDETERMINED` cases at generation, resolved at audit; the `apply-coupon` / `coupon-usage` two-endpoint split; the `A3-S3-15` wrong prediction). These were *not* corrected before generation — they are what the audit stage exists to catch, and did.

The lightest category was pure documentation/summarization tasks (`CLAUDE.md`, the CI report narrative) — accepted with no correction, because the source of truth was directly available and unambiguous.